

Inteligencja obliczeniowa w analizie danych | Inżynieria i Analiza Danych

Projekt Sieci Neuronowe

Mateusz Bugdol

Nr indeksu: 419719

Grupa ćwiczeniowa: 1

1. Cel projektu

Głównym celem projektu jest stworzenie inteligentnego systemu zdolnego do automatycznej kategoryzacji odpadów na podstawie zdjęć. Zastosowanie technik głębokiego uczenia (sieci konwolucyjne) ma na celu wsparcie procesu recyklingu poprzez automatyzację sortowania śmieci.

2. Opis zadania

Zadanie polega na wytrenowaniu sieci neuronowej typu CNN (Convolutional Neural Network) na odpowiednio zbalansowanym i zaugumentowanym zbiorze danych zawierającym zdjęcia odpadów podzielonych na 6 klas: cardboard, glass, metal, paper, plastic, trash. W ramach zadania przeprowadzono przygotowanie i podział danych, budowę architektury sieci, jej uczenie oraz ocenę wydajności za pomocą wykresów i macierzy pomyłek.

3. Implementacja sieci neuronowej CNN

3.1. Załadowanie bibliotek

```
In [1]: import os  
import numpy as np
```

```
from PIL import Image, ImageOps, ImageEnhance, ImageFont
from sklearn.model_selection import train_test_split
import tensorflow as tf
from tensorflow import keras
from tensorflow.keras import layers, models
from tensorflow.keras.callbacks import EarlyStopping, ReduceLROnPlateau, ModelCheckpoint
from sklearn.metrics import classification_report, confusion_matrix
import matplotlib.pyplot as plt
import random
import visualekera
from collections import defaultdict
from scipy.ndimage import rotate
import seaborn as sns
import json

random.seed(42)
np.random.seed(42)
tf.random.set_seed(42)
```

3.2. Stałe wartości

Powyższe stałe definiują kluczowe parametry konfiguracyjne, takie jak nazwy kategorii odpadów, ścieżka do danych, ustandaryzowany rozmiar zdjęć wejściowych i wielkość paczki (batch size). Zgrupowanie ich na początku ułatwia zarządzanie kodem i szybkie wprowadzanie modyfikacji w modelu.

```
In [2]: CLASSES = ["cardboard", "glass", "metal", "paper", "plastic", "trash"]
DATA_DIR = "dataset-resized"
IMG_SIZE = (224, 224)
BATCH_SIZE = 32
```

3.3. Pobieranie ścieżek do plików

Ten fragment kodu przechodzi przez foldery przypisane poszczególnym kategoriom odpadów i tworzy dwie listy: jedną z pełnymi ścieżkami do wszystkich zdjęć, a drugą z odpowiadającymi im numerycznymi etykietami. Przygotowuje to zbiór danych do dalszego podziału i późniejszego treningu sieci neuronowej.

```
In [3]: paths, labels = [], []
for idx, cls in enumerate(CLASSES):
```

```
folder = os.path.join(DATA_DIR, cls)
for fname in sorted(os.listdir(folder)):
    paths.append(os.path.join(folder, fname))
    labels.append(idx)
```

3.4. Podział na zbiór testowy, treningowy i walidacyjny

Ten fragment kodu dzieli cały zbiór danych na trzy części: treningową (60%), walidacyjną (20%) i testową (20%), wykonując to w dwóch krokach. Pierwsze wywołanie funkcji wydziela 60% danych do uczenia modelu (X_train), a pozostałe 40% zapisuje jako zbiór tymczasowy (X_tmp). Drugie wywołanie dzieli ten zbiór tymczasowy na równe połowy, tworząc ostateczny zbiór walidacyjny (X_val) oraz testowy (X_test). Użycie parametru stratify jest tutaj kluczowe, ponieważ gwarantuje, że w każdym z tych trzech podzbiorów zostaną zachowane równe proporcje zdjęć dla wszystkich kategorii odpadów, natomiast random_state=42 zapewnia, że ten losowy podział będzie identyczny przy każdym uruchomieniu notatnika.

```
In [4]: X_train, X_tmp, y_train, y_tmp = train_test_split(
        paths, labels, test_size=0.4, random_state=42, stratify=labels)
        X_val, X_test, y_val, y_test = train_test_split(
        X_tmp, y_tmp, test_size=0.5, random_state=42, stratify=y_tmp)
```

```
In [5]: print(f"Train: {len(X_train)}\nVal: {len(X_val)}\nTest: {len(X_test)}")
```

Train: 1516

Val: 505

Test: 506

3.5. Augumentacja zbioru treningowego

Powyższa funkcja realizuje proces sztucznego powiększania zbioru treningowego (augmentację), generując dla każdego zdjęcia zadaną liczbę nowych, zmodyfikowanych próbek. Wykorzystuje do tego losowe przekształcenia geometryczne i fotometryczne, takie jak odbicia lustrzane, obroty oraz zmiany jasności i kontrastu. Zabieg ten uodparnia model na zmienne warunki wizualne, co znacząco poprawia jego zdolność generalizacji i zapobiega zjawisku przetrenowania.

```
In [6]: def augment_dataset(paths, labels, output_dir, multiplier=3):
        os.makedirs(output_dir, exist_ok=True)
        new_paths, new_labels = list(paths), list(labels)
        for i, (path, label) in enumerate(zip(paths, labels)):
            img = Image.open(path).convert("RGB")
```

```

cls = CLASSES[label]
for j in range(multiplier):
    aug = img.copy()
    if random.random() > 0.5:
        aug = ImageOps.mirror(aug)
    if random.random() > 0.5:
        aug = ImageOps.flip(aug)

    aug_arr = np.array(aug)
    angle = random.randint(-20, 20)
    aug_arr = rotate(aug_arr, angle, reshape=False, mode='nearest')
    aug = Image.fromarray(aug_arr)

    aug = ImageEnhance.Brightness(aug).enhance(random.uniform(0.7, 1.3))
    aug = ImageEnhance.Contrast(aug).enhance(random.uniform(0.8, 1.2))
    fpath = os.path.join(output_dir, f"{cls}_aug_{i}_{j}.jpg")
    aug.save(fpath)
    new_paths.append(fpath)
    new_labels.append(label)
    if i % 100 == 0:
        print(f"  augmentacja: {i}/{len(paths)}")
return new_paths, new_labels

```

```

In [7]: if not os.path.exists("augmented"):
        X_train_aug, y_train_aug = augment_dataset(X_train, y_train, output_dir="augmented", multiplier=3)
    else:
        X_train_aug = [os.path.join("augmented", f) for f in sorted(os.listdir("augmented"))]
        y_train_aug = []
        for path in X_train_aug:
            cls = os.path.basename(path).split("_")[0]
            y_train_aug.append(CLASSES.index(cls))
        print(f"Train po augmentacji: {len(X_train_aug)}")

```

Train po augmentacji: 4548

3.6. Generator do wywoływania w trakcie treningu

Zdefiniowany generator zapewnia iteracyjne dostarczanie porcji danych do modelu podczas treningu, co optymalizuje zużycie pamięci operacyjnej poprzez ładowanie zdjęć w mniejszych paczkach. Odpowiada on również za bieżące przetwarzanie obrazów, takie jak przetasowanie próbek, zmiana rozmiaru oraz normalizacja wartości pikseli przed przekazaniem ich do sieci.

```
In [8]: def generator(paths, labels, batch_size=BATCH_SIZE):  
        while True:  
            idx = np.arange(len(paths))  
            np.random.shuffle(idx)  
            for start in range(0, len(paths), batch_size):  
                batch_idx = idx[start:start + batch_size]  
                images = []  
                for i in batch_idx:  
                    img = Image.open(paths[i]).convert("RGB").resize(IMG_SIZE)  
                    images.append(np.array(img) / 255.0)  
                yield np.array(images), np.array([labels[i] for i in batch_idx])
```

```
In [9]: train_gen = generator(X_train_aug, y_train_aug)
```

3.7. Struktura CNN

Warstwy splotowe (Conv2D): Ich zadaniem jest ekstrakcja kluczowych cech ze zdjęć, takich jak krawędzie, kształty czy tekstury, charakterystyczne dla poszczególnych klas odpadów.

Batch Normalization: Technika ta normalizuje aktywacje wewnątrz sieci, co przyspiesza proces zbieżności modelu podczas uczenia i działa delikatnie regularyzująco, zmniejszając ryzyko przetrenowania.

Funkcja aktywacji ReLU: Wprowadza nieliniowość do modelu, pozwalając mu na naukę bardziej złożonych wzorców.

MaxPooling2D: Służy do redukcji wymiarowości (ang. downsampling), zachowując jednocześnie najważniejsze wyekstrahowane cechy. Zmniejsza to liczbę parametrów w sieci i koszty obliczeniowe.

Warstwy gęste (Dense) i Dropout: Po spłaszczeniu (Flatten) danych zebranych z warstw splotowych, sieć wykorzystuje warstwy w pełni połączone do ostatecznej klasyfikacji. Dodano warstwę Dropout, która losowo wyłącza część neuronów podczas treningu, skutecznie

zapobiegając nadmiernemu dopasowaniu do danych treningowych (overfitting). Ostatnia warstwa z funkcją softmax zwraca prawdopodobieństwa przynależności do 6 klas odpadów.

```
In [10]: cnn = models.Sequential([
    layers.Conv2D(32, (3,3), padding="same", input_shape=(IMG_SIZE[0], IMG_SIZE[1], 3)),
    layers.BatchNormalization(),
    layers.Activation("relu"),
    layers.MaxPooling2D((2,2)),

    layers.Conv2D(64, (3,3), padding="same"),
    layers.BatchNormalization(),
    layers.Activation("relu"),
    layers.MaxPooling2D((2,2)),

    layers.Conv2D(128, (3,3), padding="same"),
    layers.BatchNormalization(),
    layers.Activation("relu"),
    layers.MaxPooling2D((2,2)),

    layers.Conv2D(256, (3,3), padding="same"),
    layers.BatchNormalization(),
    layers.Activation("relu"),
    layers.MaxPooling2D((2,2)),

    layers.GlobalAveragePooling2D(),
    layers.Dense(256, activation="relu", kernel_regularizer=tf.keras.regularizers.l2(0.001)),
    layers.BatchNormalization(),
    layers.Dropout(0.5),
    layers.Dense(len(CLASSES), activation="softmax")
])
```

3.8. Wizualizacja CNN

```
In [11]: font = ImageFont.truetype("arial.ttf", 32)

color_map = defaultdict(dict)
color_map[layers.Conv2D]['fill'] = '#3498db'
color_map[layers.BatchNormalization]['fill'] = '#f1c40f'
color_map[layers.Activation]['fill'] = '#e67e22'
```

```

color_map[layers.MaxPooling2D]['fill'] = '#e74c3c'
color_map[layers.GlobalAveragePooling2D]['fill'] = '#9b59b6'
color_map[layers.Dense]['fill'] = '#2ecc71'
color_map[layers.Dropout]['fill'] = '#95a5a6'

img = visualkeras.layered_view(
    cnn,
    legend=True,
    font=font,
    spacing=30,
    color_map=color_map,
    scale_xy=2,
    scale_z=1
)

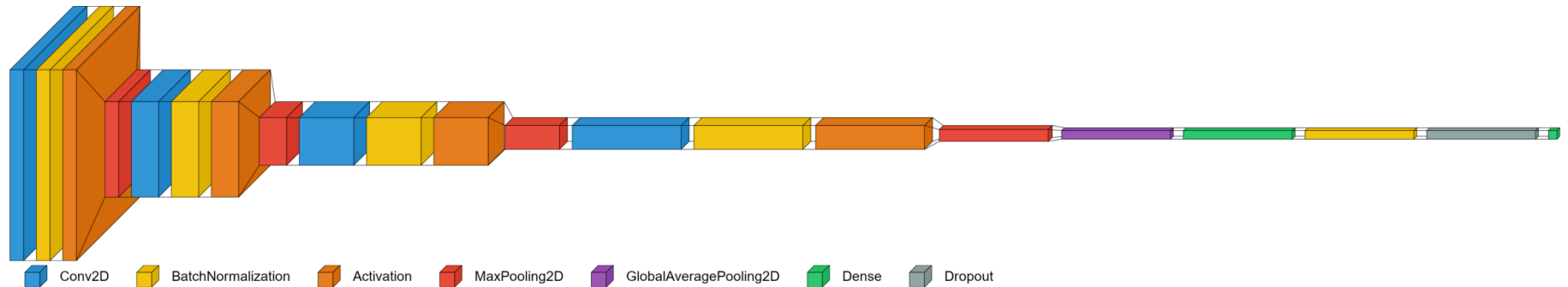
fig, ax = plt.subplots(figsize=(24, 12))
fig.patch.set_facecolor('white')
ax.imshow(img)
ax.axis('off')
fig.suptitle("Architektura CNN dla klasyfikacji odpadów", fontsize=28, fontweight='bold', y=0.7)
plt.tight_layout()
plt.show()

```

p:\Anaconda\envs\cnn\lib\site-packages\visualkeras\layered.py:231: UserWarning: The legend_text_spacing_offset parameter is deprecated and will be removed in a future release.

warnings.warn("The legend_text_spacing_offset parameter is deprecated and will be removed in a future release.")

Architektura CNN dla klasyfikacji odpadów



Powyższy schemat przedstawia architekturę zaprojektowanej sieci konwolucyjnej, ukazując przepływ danych przez jej kolejne, sekwencyjnie ułożone warstwy. Zobrazowano na nim drogę od wejścia obrazu, poprzez bloki ekstrakcji cech (warstwy plotowe, normalizacyjne i łączące), aż

po końcowe warstwy gęste odpowiedzialne za ostateczną klasyfikację do jednej z 6 kategorii. Taka wizualizacja ułatwia szybkie zrozumienie struktury modelu oraz pozwala prześledzić zmiany wymiarowości tensorów na każdym etapie przetwarzania.

3.9. Trenowanie modelu

Proces kompilacji sieci konwolucyjnej zrealizowano z wykorzystaniem optymalizatora Adam o zredukowanym współczynniku uczenia, funkcji straty sparse categorical crossentropy oraz metryki dokładności. W celu optymalizacji czasu obliczeń zaimplementowano mechanizm warunkowego ładowania gotowych, wcześniej wyuczonych wag modelu. W przypadku inicjalizacji nowego treningu, proces ten jest ściśle kontrolowany przez trzy mechanizmy zabezpieczające: wczesne zatrzymywanie chroniące przed zjawiskiem przetrenowania, dynamiczną redukcję współczynnika uczenia w momentach stagnacji oraz zapis wyłącznie najlepszej iteracji sieci. Samo uczenie odbywa się iteracyjnie z użyciem generatora danych przez maksymalnie 100 epok, przy jednoczesnej ciągłej ewaluacji wyników na wydzielonym zbiorze walidacyjnym. Po zakończeniu operacji cała historia postępów jest ostatecznie eksportowana do pliku JSON, co umożliwia jej późniejszą analizę oraz wizualizację na wykresach.

```
In [12]: cnn.compile(
    optimizer=keras.optimizers.Adam(learning_rate=0.0001, clipnorm=1.0),
    loss="sparse_categorical_crossentropy",
    metrics=["accuracy"]
)

In [13]: if os.path.exists("best_model.keras"):
    print("Ładowanie najlepszego modelu z pliku...")
    cnn.load_weights("best_model.keras")
    with open('history.json', 'r') as f:
        history_dict = json.load(f)
else:
    callbacks = [
        EarlyStopping(monitor="val_loss", patience=10, restore_best_weights=True),
        ReduceLROnPlateau(monitor="val_loss", factor=0.5, patience=5, min_lr=1e-7),
        ModelCheckpoint("best_model.keras", monitor="val_accuracy", save_best_only=True)
    ]

    X_val_arr = np.array([
        np.array(Image.open(p).convert("RGB").resize(IMG_SIZE)) / 255.0
        for p in X_val
    ])
    y_val_arr = np.array(y_val)
```



```
history = cnn.fit(
    train_gen,
    steps_per_epoch = len(X_train_aug) // BATCH_SIZE,
    epochs           = 100,
    validation_data   = (X_val_arr, y_val_arr),
    validation_batch_size = 64,
    callbacks         = callbacks
)
history_dict = {key: [float(val) for val in values] for key, values in history.history.items()}

with open('history.json', 'w') as f:
    json.dump(history_dict, f)
```

Ładowanie najlepszego modelu z pliku...

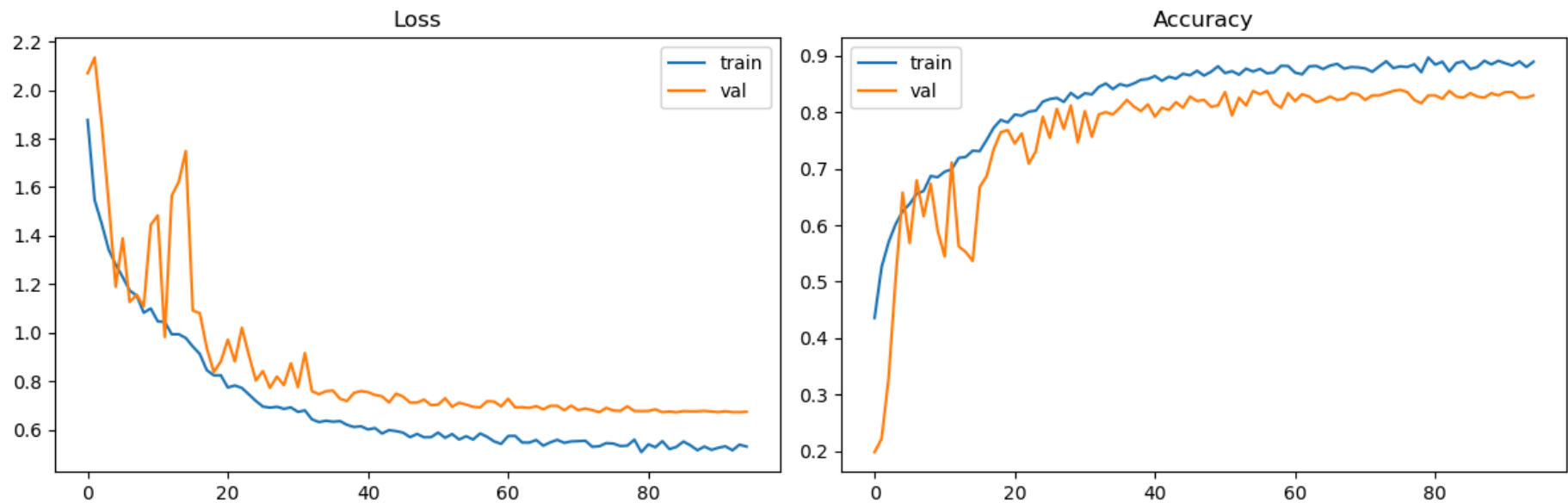
3.10. Wykresy Loss i Accuracy

```
In [14]: fig, (ax1, ax2) = plt.subplots(1, 2, figsize=(12, 4))

ax1.plot(history_dict["loss"], label="train")
ax1.plot(history_dict["val_loss"], label="val")
ax1.set_title("Loss")
ax1.legend()

ax2.plot(history_dict["accuracy"], label="train")
ax2.plot(history_dict["val_accuracy"], label="val")
ax2.set_title("Accuracy")
ax2.legend()

plt.tight_layout()
plt.show()
```



Z wykresów Accuracy można odczytać, że skuteczność modelu systematycznie rośnie z każdą epoką, zarówno dla zbioru treningowego, jak i walidacyjnego. Jednocześnie wykres Loss (funkcja straty) pokazuje stabilny spadek. Brak znacznej rozbieżności między krzywymi treningowymi i walidacyjnymi świadczy o dobrym uogólnianiu wiedzy przez model.

3.11. Confusion Matrix

```
In [15]: X_test_arr = np.array([
    np.array(Image.open(p).convert("RGB").resize(IMG_SIZE)) / 255.0
    for p in X_test
])
y_test_arr = np.array(y_test)

loss, acc = cnn.evaluate(X_test_arr, y_test_arr, verbose=1)
print(f"Test accuracy: {acc:.2%}")

y_pred = np.argmax(cnn.predict(X_test_arr), axis=1)
print(classification_report(y_test_arr, y_pred, target_names=CLASSES))
```

16/16 [=====] - 4s 25ms/step - loss: 0.7131 - accuracy: 0.8221

Test accuracy: 82.21%

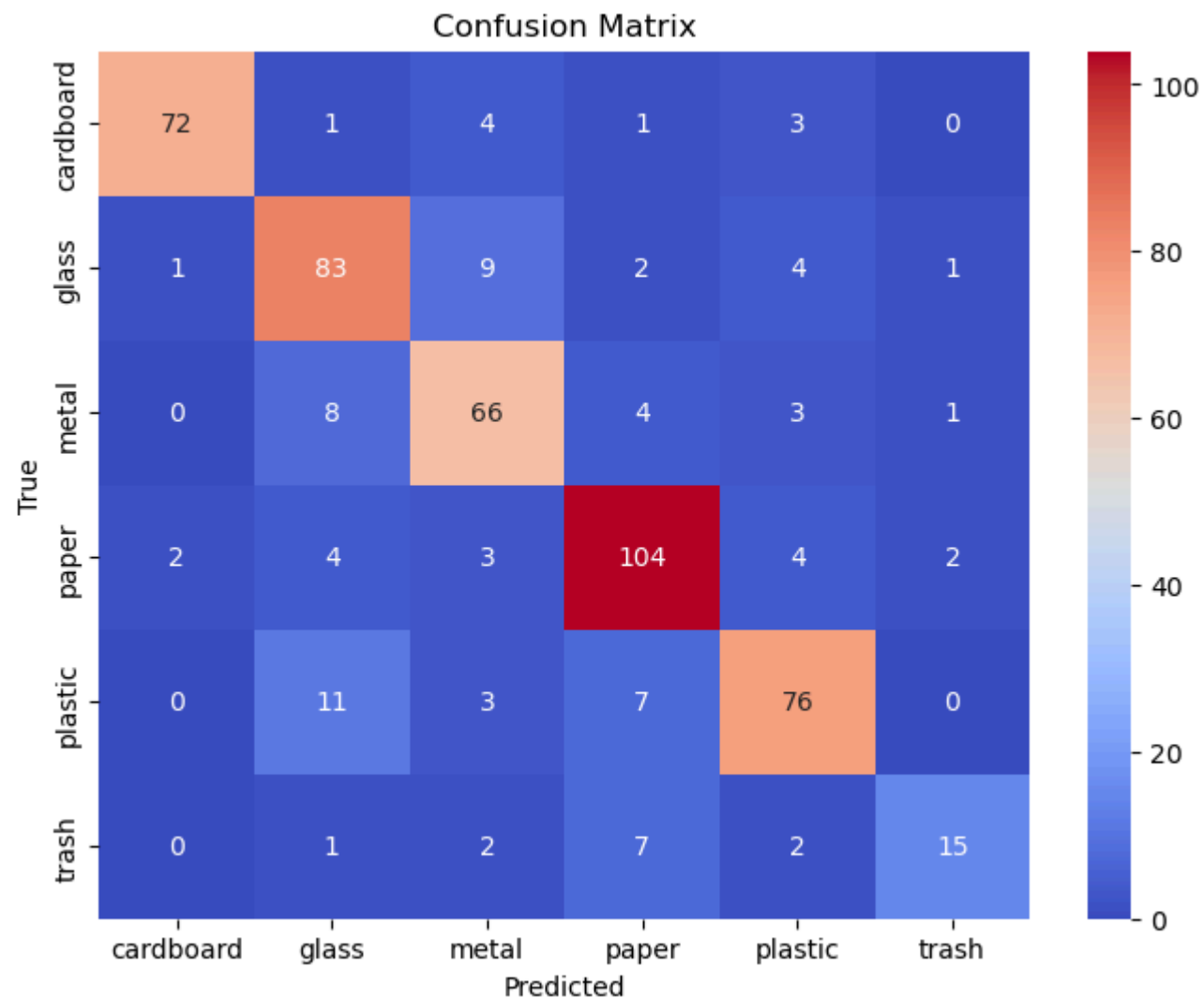
16/16 [=====] - 0s 19ms/step

	precision	recall	f1-score	support
cardboard	0.96	0.89	0.92	81
glass	0.77	0.83	0.80	100
metal	0.76	0.80	0.78	82
paper	0.83	0.87	0.85	119
plastic	0.83	0.78	0.80	97
trash	0.79	0.56	0.65	27
accuracy			0.82	506
macro avg	0.82	0.79	0.80	506
weighted avg	0.82	0.82	0.82	506

Model osiągnął na zbiorze testowym ogólną dokładność (accuracy) na poziomie 82,21%, co potwierdza jego dobrą skuteczność w rozpoznawaniu badanych kategorii. Analiza szczegółowych metryk wskazuje, że sieć najlepiej radzi sobie z klasyfikacją tektury (cardboard), osiągając dla niej najwyższy wskaźnik F1-score równy 0,92. Z kolei największe trudności sprawia kategoria odpadów mieszanych (trash), gdzie niska czułość (56%) sugeruje problemy modelu z poprawną identyfikacją tej wysoce zróżnicowanej wizualnie grupy.

```
In [16]: cm = confusion_matrix(y_test_arr, y_pred)
plt.figure(figsize=(8,6))
sns.heatmap(cm, annot=True, fmt="d", cmap="coolwarm", xticklabels=CLASSES, yticklabels=CLASSES)
plt.xlabel("Predicted")
plt.ylabel("True")
plt.title("Confusion Matrix")
```

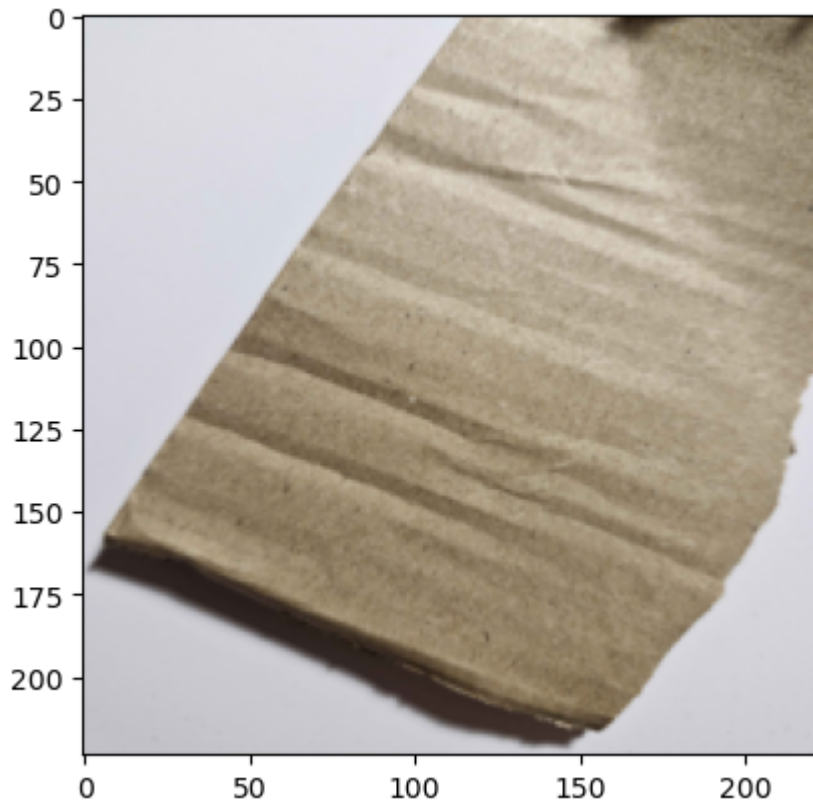
```
Out[16]: Text(0.5, 1.0, 'Confusion Matrix')
```



Przedstawiona macierz pomyłek potwierdza dobrą skuteczność modelu, co obrazuje wyraźne skupienie większości poprawnych predykcji na głównej przekątnej. Największą trafnością charakteryzuje się klasyfikacja tektury (cardboard), podczas gdy najwięcej błędów rozproszenia generuje kategoria odpadów mieszanych (trash), co wynika z jej dużej różnorodności wizualnej. Widoczne są również sporadyczne pomyłki pomiędzy materiałami o zbliżonym wyglądzie, takimi jak szkło (glass) i plastik (plastic).

3.12. Przykładowe predykcje klas

```
In [17]: karton = Image.open("karton.jpg").convert("RGB").resize(IMG_SIZE)
plt.imshow(karton)
plt.show()
```

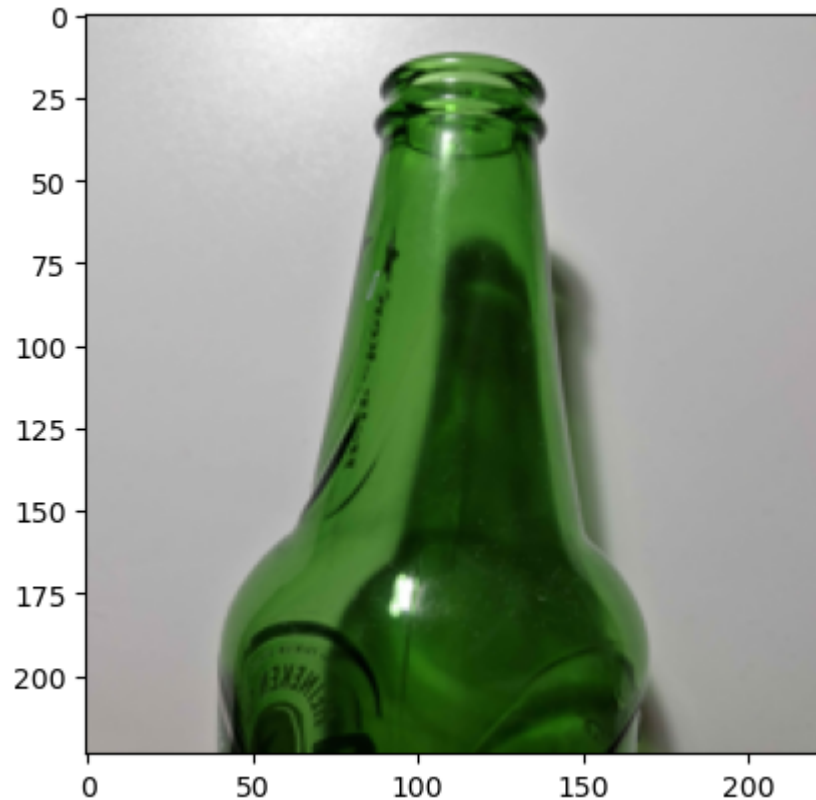


```
In [18]: karton_arr = np.array(karton) / 255.0
pred = cnn.predict(karton_arr[np.newaxis, ...])
pred_class = CLASSES[np.argmax(pred)]
print(f"Predykcja dla karton.jpg: {pred_class} (prawdopodobieństwo: {np.max(pred):.2%})")
```

```
1/1 [=====] - 0s 39ms/step
Predykcja dla karton.jpg: cardboard (prawdopodobieństwo: 76.99%)
```

```
In [19]: szklo = Image.open("szklo.jpg").convert("RGB").resize(IMG_SIZE)
plt.imshow(szklo)
```

```
Out[19]: <matplotlib.image.AxesImage at 0x2ba3872b040>
```

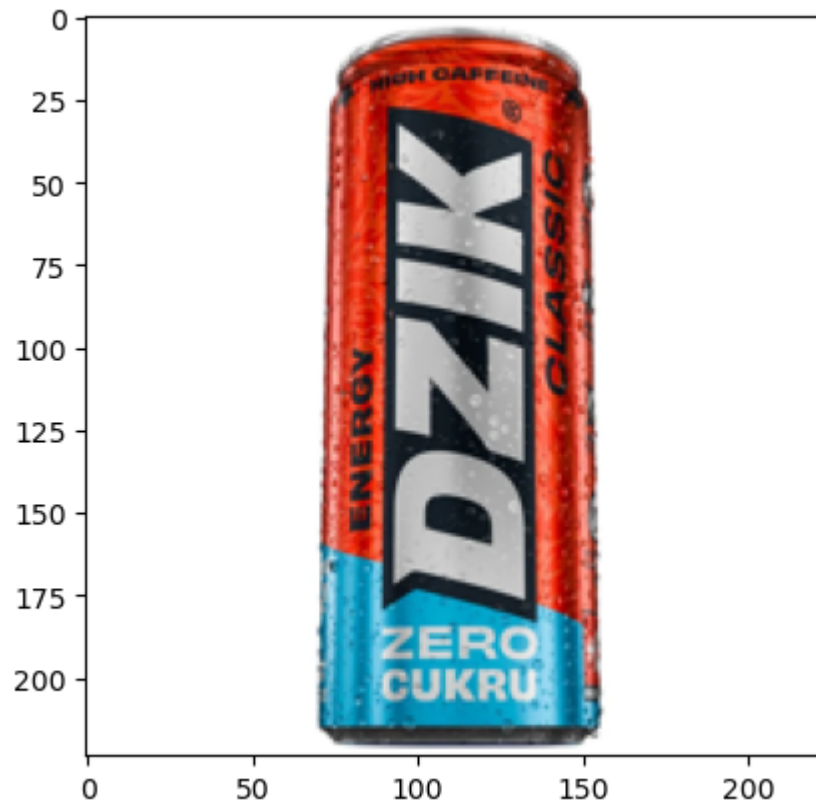


```
In [20]: szklo_arr = np.array(szklo) / 255.0
pred = cnn.predict(szklo_arr[np.newaxis, ...])
pred_class = CLASSES[np.argmax(pred)]
print(f"Predykcja dla szklo.jpg: {pred_class} (prawdopieństwo: {np.max(pred):.2%})")
```

```
1/1 [=====] - 0s 19ms/step
Predykcja dla szklo.jpg: glass (prawdopieństwo: 98.23%)
```

```
In [21]: puszka_kolor = Image.open("puszka_kolor.jpg").convert("RGB").resize(IMG_SIZE)
plt.imshow(puszka_kolor)
```

Out[21]: <matplotlib.image.AxesImage at 0x2ba386d6050>



```
In [22]: puszka_kolor_arr = np.array(puszka_kolor) / 255.0
pred = cnn.predict(puszka_kolor_arr[np.newaxis, ...])
print(f"Predicted class: {CLASSES[np.argmax(pred)]} (confidence: {np.max(pred):.2%})")
```

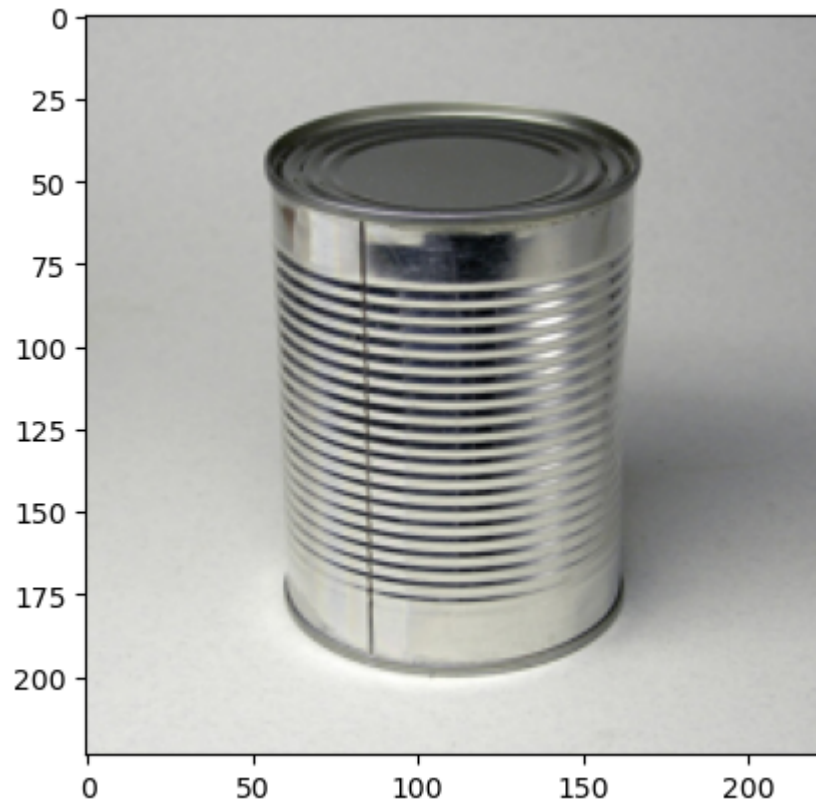
1/1 [=====] - 0s 20ms/step
 Predicted class: paper (confidence: 84.94%)

Błędna klasyfikacja kolorowej puszkii jako papieru z pewnością 84,94% wynika najprawdopodobniej z braku odpowiedniej różnorodności w zbiorze treningowym. W danych uczących kategorii metalu przeważały klasyczne, silnie metaliczne i odbłaskowe obiekty, przez co model nie wykształcił zdolności poprawnego rozpoznawania puszek pokrytych w całości barwnym, matowym nadrukiem.

```
In [23]: puszka_biala = Image.open("puszka_biala.jpg").convert("RGB").resize(IMG_SIZE)
```

```
plt.imshow(puszka_biala)
```

Out[23]: <matplotlib.image.AxesImage at 0x2ba38c8eb00>



```
In [24]: puszka_biala_arr = np.array(puszka_biala) / 255.0  
pred = cnn.predict(puszka_biala_arr[np.newaxis, ...])  
print(f"Predicted class: {CLASSES[np.argmax(pred)]} (confidence: {np.max(pred):.2%})")
```

1/1 [=====] - 0s 18ms/step
Predicted class: metal (confidence: 96.83%)

```
In [25]: papier = Image.open("papier.jpg").convert("RGB").resize(IMG_SIZE)  
plt.imshow(papier)
```

Out[25]: <matplotlib.image.AxesImage at 0x2ba38d24130>

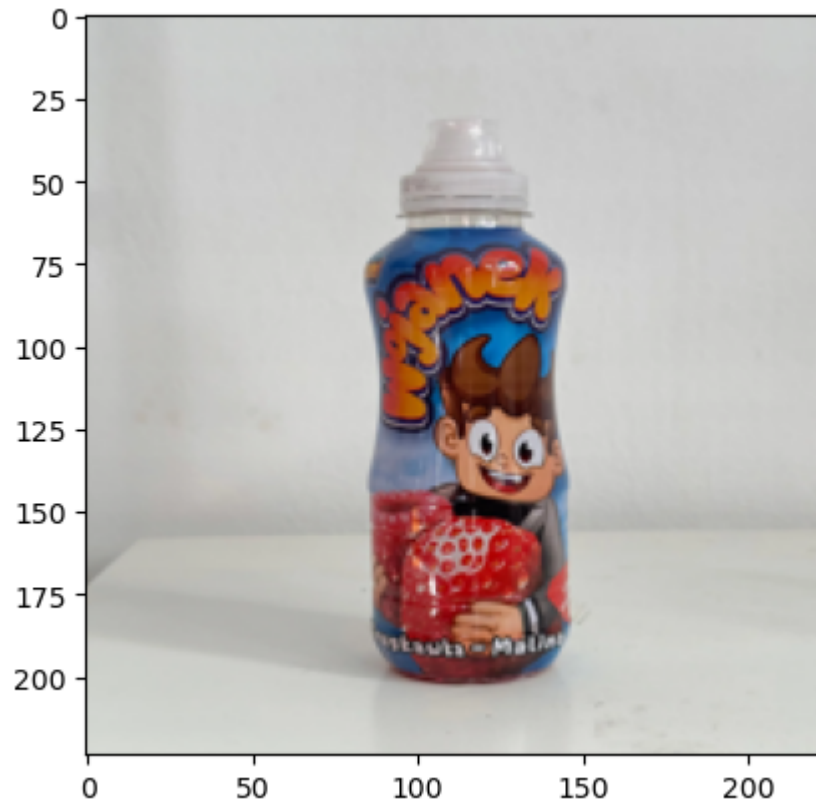


```
In [26]: papier_arr = np.array(papier) / 255.0
pred = cnn.predict(papier_arr[np.newaxis, ...])
print(f"Predicted class: {CLASSES[np.argmax(pred)]} (confidence: {np.max(pred):.2%})")
```

```
1/1 [=====] - 0s 19ms/step
Predicted class: paper (confidence: 89.71%)
```

```
In [27]: plastik = Image.open("plastik.jpg").convert("RGB").resize(IMG_SIZE)
plt.imshow(plastik)
```

```
Out[27]: <matplotlib.image.AxesImage at 0x2ba38ed9a80>
```



```
In [28]: plastik_arr = np.array(plastik) / 255.0
pred = cnn.predict(plastik_arr[np.newaxis, ...])
pred_class = CLASSES[np.argmax(pred)]
print(f"Predykcja dla plastik.jpg: {pred_class} (prawdopodobieństwo: {np.max(pred):.2%})")
```

```
1/1 [=====] - 0s 21ms/step
Predykcja dla plastik.jpg: plastic (prawdopodobieństwo: 58.73%)
```

4. Podsumowanie

W ramach projektu z powodzeniem zaprojektowano i wytrenowano konwolucyjną sieć neuronową (CNN) przeznaczoną do automatycznej klasyfikacji sześciu kategorii odpadów. Zastosowanie technik takich jak augmentacja obrazów oraz mechanizmy optymalizacyjne chroniące przed przetrenowaniem pozwoliło modelowi osiągnąć zadowalającą skuteczność na poziomie ponad 82% na zbiorze testowym. Choć sieć

wykazuje wysoką trafność w rozpoznawaniu jednorodnych materiałów (np. tekstura czy papier), analiza błędów ujawniła trudności przy klasyfikacji obiektów nietypowych wizualnie oraz odpadów mieszanych. Dalszy rozwój systemu i poprawa jego dokładności wymagałyby przede wszystkim rozbudowy zbioru treningowego o większą liczbę zróżnicowanych próbek i trudnych przypadków brzegowych.